PRACTICA 2

Lilia Alejandra Padilla Hernández
Expediente: 673256

DESARROLLO EN LA NUBE
Maestra:Marcela A. Rosales

14/03/2026

Descripción del Problema

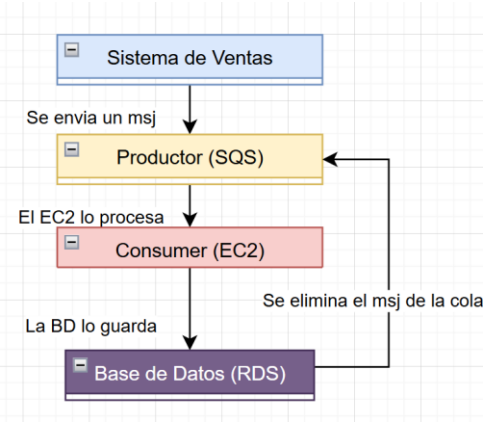
Nombre de la aplicación:

Coffee orders

Problema que busca resolver:
En el contexto de una cafetería con alta demanda de pedidos durante ciertas horas del día, se enfrenta a la saturación del sistema de caja si los pedidos no se gestionan adecuadamente. Esto los lleva a la pérdida de pedidos y desorganización en el proceso de preparación ya que no se tiene un apoyo de cómo fueron llegando los pedidos. Para mitigar este problema, se propone una solución basada en colas de mensajes para gestionar los pedidos de manera asíncrona y evitar la sobrecarga. Cada pedido es procesado por un barista conforme su capacidad lo permita, evitando la congestión en el sistema y garantizando que ningún pedido se pierda.

Este sistema se complementa con una base de datos para llevar un registro permanente de cada orden, donde se almacena el tipo de café, el momento de la orden y el estado de la misma. El objetivo es asegurar una gestión eficiente de los pedidos, permitiendo a los baristas procesar las órdenes sin saturar el sistema y manteniendo un registro estructurado para futuras consultas.

Diagrama de la Arquitectura



El diagrama de arquitectura refleja cómo se interconectan los servicios de **AWS** con la aplicación en **EC2** para gestionar los pedidos de café en la cafetería.

1. **Amazon SQS:**
   El sistema de venta de la cafetería (el productor) genera mensajes con el tipo de café y el timestamp, los cuales son enviados a la cola **SQS**. Esto permite manejar la sobrecarga de órdenes durante horas de alta demanda.

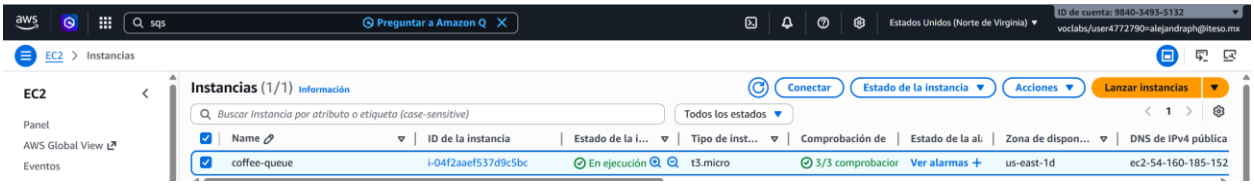
2. **Instancia EC2 (Consumer):**
   La instancia **EC2** se encarga de ejecutar el **consumidor**, que realiza un polling infinito sobre la cola **SQS**. Cuando hay mensajes disponibles, la aplicación en EC2 procesa cada mensaje, extrae el tipo de café y el timestamp, y los inserta en la base de datos **RDS**. Posteriormente, el mensaje es eliminado de la cola **SQS** para evitar duplicación de la orden.

3. **Amazon RDS:**
   La base de datos **RDS** es utilizada para almacenar todos los registros de las órdenes de café. Cada entrada en la tabla **coffee_orders** contiene un **id**, el **timestamp** de la orden, el **tipo de café** y el **estado** de la orden (en este caso, siempre "created" cuando se ingresa a la base de datos).

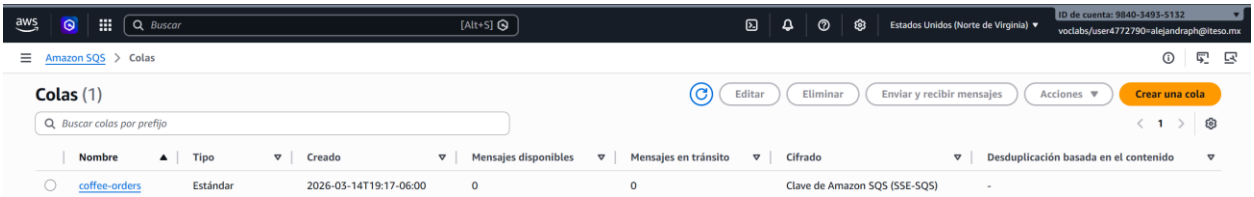
4. **Interacción de los Componentes:**
   El flujo de datos comienza cuando el productor envía un mensaje a **SQS**. Este mensaje es recibido por la instancia **EC2** que, luego de procesarlo, almacena la información en **RDS**. Al completar este proceso, el mensaje se elimina de la cola. Esto permite que el sistema sea escalable, eficiente y que ningún pedido se pierda, incluso si hay una alta carga de trabajo en la cafetería.

Lo que hice:

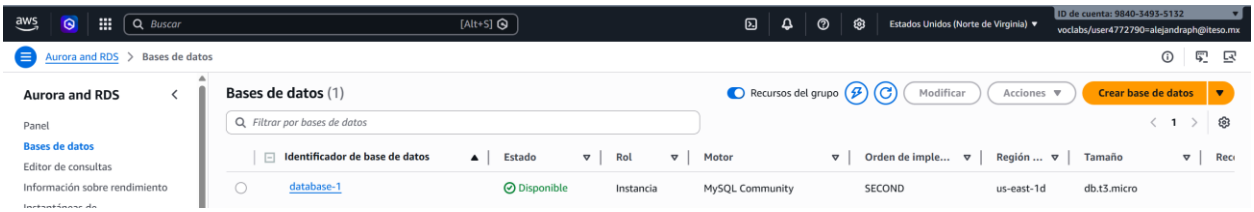
Crear la instancia y configurarla directo.



Crear la SQS y configurarla:



Crear la DB que me costó muchísimo que me dejara crearla ya que me mencionaba que no tenía los permisos, no me dejo ni con Aurora, ni con Postgrest pero al fin me dejo con MYSQL community.



Una vez que cree todo, intenté conectarme a la BD para poder crear la tabla, pero no me dejaba conectar.

Me conecte con la instancia, inicie el env e instale todo las dependencias para poder cargar mi programa, lo subí al EC2, y lo corrí. Pero como no se podía conectar a la BD no me dejaba proseguir, y aquí ya no pude continuar.

De verdad que lo estuve intentando pero no logré que se pudiera conectar la base de datos, y no supe cómo arreglarlo.

Repositorio:

https://github.com/LiliDYue/Practica2_queueCoffee